Weights Update During Backpropagation: Better Convergence for Scale Invariant Networks in Specific Scenarios

Aleksandr Shpilevoi 3773516 ms53dumu@studserv.uni-leipzig.de
Earth System Data Science and Remote Sensing, Leipzig University

September 9, 2025

Abstract

Parameter optimization in artificial neural networks is powered by the chain rule, as a neural network is equivalent to a nested function. The conventional approach to update weights is to calculate all $\frac{dLoss}{d\text{Weight}}$ derivatives and only then perform the update of the weights. In this approach, Weights, which were used during a forward pass, are also employed for forwarding dLoss to the previous layer. However, there were no experiments for the setup: updating the Weights immediately after obtaining $\frac{dLoss}{d\text{Weight}}$ and using just the newly updated Weights in the chain rule. The unconventional online setup *could be* similar to Nesterov Momentum, which enhances an optimization process or can be a noise injector, which also can improve training. Here we show that the unconventional approach can give significantly better convergence, albeit only in a small set of cases. The new approach gives up to 4% accuracy gain for fully connected networks with batch normalization on simple datasets like MNIST. Our results demonstrate how the unconventional weight update approach influences MLPs on different training setups and datasets. We anticipate the present essay to be a starting point for more analytical and mathematically strict research on this topic: the approach differs from the dominant algorithm but may lead to a significant improvement for specific scenarios. Additionally, there is a lot of space for further experiments. E.g., using the online update method on certain epochs, specific depth, or with probability.

1 Introduction

Neural networks excel in many tasks. In computer vision, they can perform object detection [1, 2], object tracking [3], instance segmentation [4], semantic segmentation [5], panoptic segmentation [6], anomaly detection [7]. In natural language processing and language modeling, translation [8, 9], text generation [10, 11], text refinement are done with neural networks too [12]. Multimodal networks can combine abilities of understanding language, images, or videos and speech: text-to-image [13, 14], image-to-text problems are solved with transformer-based approaches [15, 16]. One can tackle an image, video, or sound generation problem with generative adversarial networks (GANs) [17, 18] or with more modern diffusion models [19, 20]. Diffusion models offer a trade-off between speed and accuracy balance in comparison with classical feed-forward models [21]. Speech recognition [22, 23], text-to-speech synthesis can also be tackled with neural networks [24, 25]. Any vector data-rich problem can be solved with neural networks [26]. This versatility arises from the modular and composable nature of neural networks [26, 27].

At the same time, any neural network mentioned before is a complex (nested) function consisting of layers (matrix operations) [28], activation functions [29, 30, 31, 32], normalization functions [33, 34, 35, 36]. The more layers we use to build a neural network, the more powerful but computationally expensive the network will be. Different tasks require different computational blocks - layers. Inference of a neural network is a feedforward process where every layer processes the output of the previous one. The shape and output of the last layer depend on a neural network task: a real number for regression, a 3 x width x height dimensional tensor for image generative models, next word prediction for LLMs, bounding boxes for object detectors, etc. So, every block of a neural net contains tunable parameters and requires an efficient way to adjust the parameters.

These building blocks of neural networks have parameters that are optimized during the training process. Every operation inside a neural net must be differentiable. To optimize any function, we first must establish a quantitative measure of the function's output - the loss function. For example, for object detection, the loss function could be intersection over union between predicted bounding box and ground truth one, for classification (predicting one class from a finite set), it can be the cross-entropy. The nested nature of the networks necessitates an algorithm to

compute gradients for complex functions.

Deep neural network optimization employs backpropagation [37]. It has linear complexity with respect to the number of layers: derivatives of loss with respect to the weights of the layer are computed for every layer in reversed order - from deepest layers to shallowest. Figure 1 shows a simple neural network for regression and shows the conventional approach to finding derivatives for nested matrix functions and the following weight update stage that uses the obtained derivatives. Over time, different techniques were proposed to improve or adapt this procedure for efficiency and better performance.

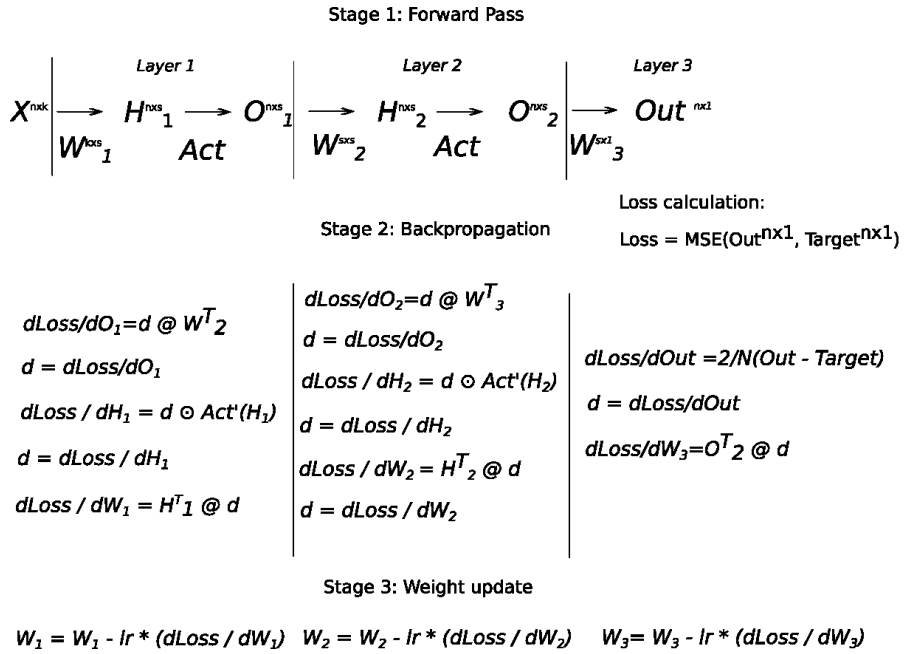


Figure 1: The traditional backpropagation and weights update approach: Firstly calculate all derivatives. Only after perform gradient decent for all weights at once

1

Several ideas have been proposed to improve weight update policies and the backpropagation algorithm [37]. The most popular approach is not to perform backpropagation and weight updates for shallow layers but only for deep ones [38] to transfer knowledge of a pre-trained network for a specific domain. Another approach motivated by the high computational cost of Transformers [39] performs several backpropagation passes, accumulates and averages gradients [40] over a few iterations, and updates the weights only afterwards. One can also inject noise into gradients to

improve convergence [41], although this approach has not become standard in deep learning. At the same time, backpropagation and its many modifications are only responsible for the gradient computation and not for the weight updating procedure.

The weight updating process can be done with various optimizers. While Figure 1 describes the simplest approach for weight update - Stochastic Gradient Descent (SGD), a lot of research has been conducted on how to improve the weight update process. AdamW [42] and Adam [43] became the default optimizers in deep learning: instead of doing the same step in all dimensions of a weights matrix, the optimizers take bigger steps in directions with less variance. Also, Adam-family optimizers calculate momentum: exponential moving average (EMA) of current and previous state of the gradient (1). RMSProp [44] is a predecessor of the Adam optimizer and uses a similar idea of scaling step size, but by the norm of a gradient for different parameters of a weights matrix. No optimizers guarantee reaching a global minimum and adequate performance, but every empirical improvement is valuable.

At the same time, deep learning remains a very heuristic area because of its nature: a very high-dimensional, nonlinear problems of approximating complex distributions. Terabytes of high-dimensional data are used during the optimization. The training process can last months. Thus, every heuristic that improves training and increases metrics is useful. Many standard techniques like learning rate scheduling [45], gradient clipping [46], and label smoothing [47] in deep learning have no strict proofs but are commonly used in practice. So, every heuristic that yields an improvement, even sub-percent gains, can become the standard and move us forward.

To find such an improvement, here we focus on the Nesterov Accelerated Gradient (NAG) [48] optimizer. NAG stands out due to the possible integration of its core idea into backpropagation and the weight update algorithm. NAG is built over momentum SGD [49]. The core idea of NAG is computing momentum of the current gradient and a gradient after a step of gradient descent (the "look-ahead" term): equation (1) below shows the classical momentum SGD; equation (2) - NAG.

$$v_t = \gamma v_{t-1} + \eta \nabla_\theta L(\theta_{t-1}) \tag{1}$$

$$v_t = \gamma v_{t-1} + \eta \nabla_\theta L\big(\theta_{t-1} - \gamma v_{t-1}\big) \tag{2}$$

However, no prior experiments have investigated a similar early update mechanism for the backpropagation algorithm: updating each layer's weights immediately after computing $\frac{dLoss}{d\text{Weight}}$, and then using the just-updated weight matrix to propagate the loss gradient to the previous layer. Figure 2 describes the proposed algorithm with zero gamma coefficient from the equation (2). Here we show that the new algorithm improves convergence of scale invariant (batch normalized) MLPs on simple datasets like MNIST in specific configurations.

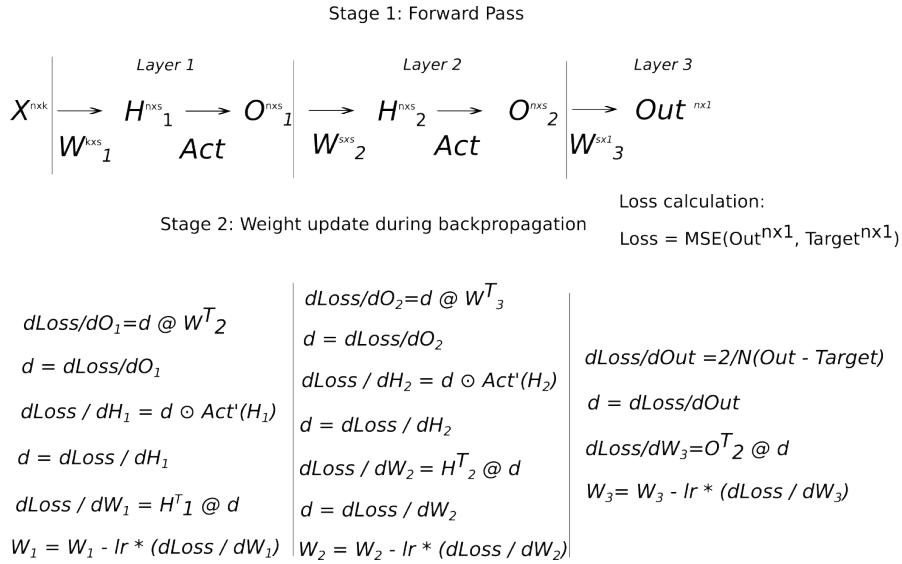


Figure 2: The proposed NAG-inspired approach: forward the gradient to the previous layer and update the weight matrix W immediately after obtaining its gradient.

2 Methods

2.1 Comparison pipeline

Initially, we evaluate an extreme configuration in which only the “look-ahead” term is used, without calculating exponential moving average (EMA) between the previous and updated weight matrices. Specifically, the momentum coefficient gamma for the previous weights is equal to zero, equation (2). We used this approach in only “look-ahead” mode to validate the concept of the "as soon as possible" updating approach. The conventional approach and the proposed online update method are compared in the following pipeline: two identical networks are initialized with the same weights. The first one is trained in the traditional way, the second one with weights update during backpropagation. The methods were evaluated in parallel, using identical setups, while varying the hyperparameters.

2.2 Hyperparameters setup

Experiments were conducted using two architectural types of fully connected multilayer perceptrons and different batch size / learning rate combinations. The first architecture type preserves input dimensionality during the forward pass (persisting / width constant architecture), while the second one subsequently reduces dimensionality to the nearest power of 2 (triangle architecture). Batch sizes $\{2^k\}_{k=1}^{10}$ were tested for 2-7 layered perceptrons. We used ReLU [29] as the activation function and batch normalization [33]: training with the new approach is unstable independent on hyperparameters without batch-norm layers. SGD without momentum was used, 0.0005/0.001 learning rates and 0/0.0001 L2 regularization terms were tested. The training lasted for 10 epochs, with a learning rate decay of 0.1 applied at epoch 7. The epoch count is determined by the accuracy metric reaching a plateau. The weights initialization is He [50] uniform, and bias is used on every layer. Experiments were conducted on MNIST [51], Fashion-MNIST [52], and CIFAR-10 [53] datasets. The accuracy metric is compared on the validation subset. Three runs with different initial weight initializations were performed with fixed seeds from 0 to 2. A standard cross-entropy served as the loss function. An RTX 2080Ti GPU with CUDA 12.2 was used. The variety of setups to test requires to use a high-performance deep learning framework.

2.3 Code implementation

All computations were implemented using PyTorch [54] leveraging its capability to create custom layers with arbitrary behavior during both the forward and backward passes. Consequently, computational overhead stays the same as for the traditional approach. Implementation of the online backpropagation method only required to modify the .backward() method for BatchNorm1d and Linear layers. Weight update operations were inserted before returning gradients from a layer. The calculations were done using resources of the Leipzig University Computing Center[1]. Source code is publicly available at GitHub repository, 800 lines of Python code.

3 Results

3.1 Overall Performance

On average across all setups, the new algorithm degrades the training process but with high deviation, Table 1 shows mean and standard deviation of accuracy gain caused by the new algorithm.

Table 1: Mean improvement and standard deviation, per dataset.

Dataset	Mean Accuracy Gain	Std Accuracy Gain
MNIST	-0.721000	3.819000
Cifar10	-0.679000	1.852000
Fashion MNIST	-0.595000	3.071000

Despite the negative average accuracy change, the weight update during backpropagation achieves accuracy improvement of more than 1% in 1.2% of the MNIST setups, 0.8% of the Fashion MNIST setups, and only 0.2% of the CIFAR-10 setups. Figure 3 shows the distribution of the accuracy changes with a strong peak at zero and negative skew. However, the proposed approach can improve the accuracy to 4% under certain conditions.

[1] https://www.sc.uni-leipzig.de/, Accessed on 15 August 2025

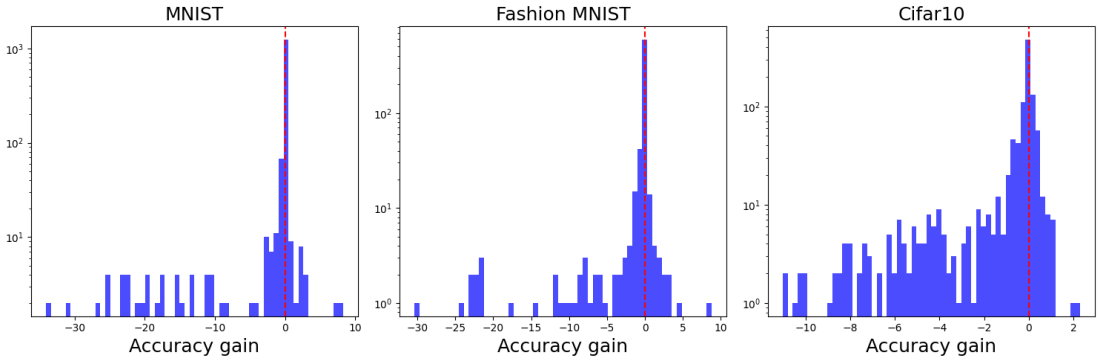


Figure 3: Distribution of metric gain variance between the proposed online weights update and classical approaches, y axis is logscale

3.2 Optimal empirical configurations

Observed conditions for higher gains are:

- Lower learning rates
- The triangle feature pass architecture
- In general, batch size less than 32

The empirically better combination of depth and batch size depend on the dataset. For MNIST and Fashion MNIST, batch sizes 2-4 for 4-5 layered networks give up to 4% gain. In contrast, 5-7 layered networks and batch sizes 8-32 give up to 0.7% gain for the CIFAR-10 dataset. Thus, data complexity influences which combination of depth and batch size will give an accuracy gain. L2 decay and batch size more than 64 do have a negligible effect on the online approach. Table 2 and Table 3 show the best and the worst configurations.

Batch	Layers	LR	L2 Decay	Architecture	Dataset	Accuracy gain mean	Std
2	5	0.0001	0.0	triangle	mnist	4.675	2.975
2	5	0.0001	0.0001	triangle	mnist	4.675	2.975
2	4	0.0001	0.0	triangle	mnist	1.272	2.77
2	4	0.0001	0.0001	triangle	mnist	1.272	2.77
2	4	0.0001	0.0	persisting	fashion mnist	1.236	1.295
32	6	0.005	0.0	triangle	CIFAR-10	0.713	0.42
8	7	0.0001	0.0	persisting	CIFAR-10	0.597	0.474
16	7	0.005	0.0	triangle	CIFAR-10	0.558	0.58
2	6	0.0001	0.0	triangle	fashion mnist	0.542	4.915
2	4	0.0001	0.0	triangle	CIFAR-10	0.48	0.01
32	6	0.005	0.0	persisting	CIFAR-10	0.463	1.152
2	6	0.0001	0.0	persisting	CIFAR-10	0.44	0.674
64	7	0.005	0.0	triangle	CIFAR-10	0.367	0.837
8	4	0.005	0.0	triangle	CIFAR-10	0.367	0.624
16	5	0.005	0.0	triangle	CIFAR-10	0.347	0.398

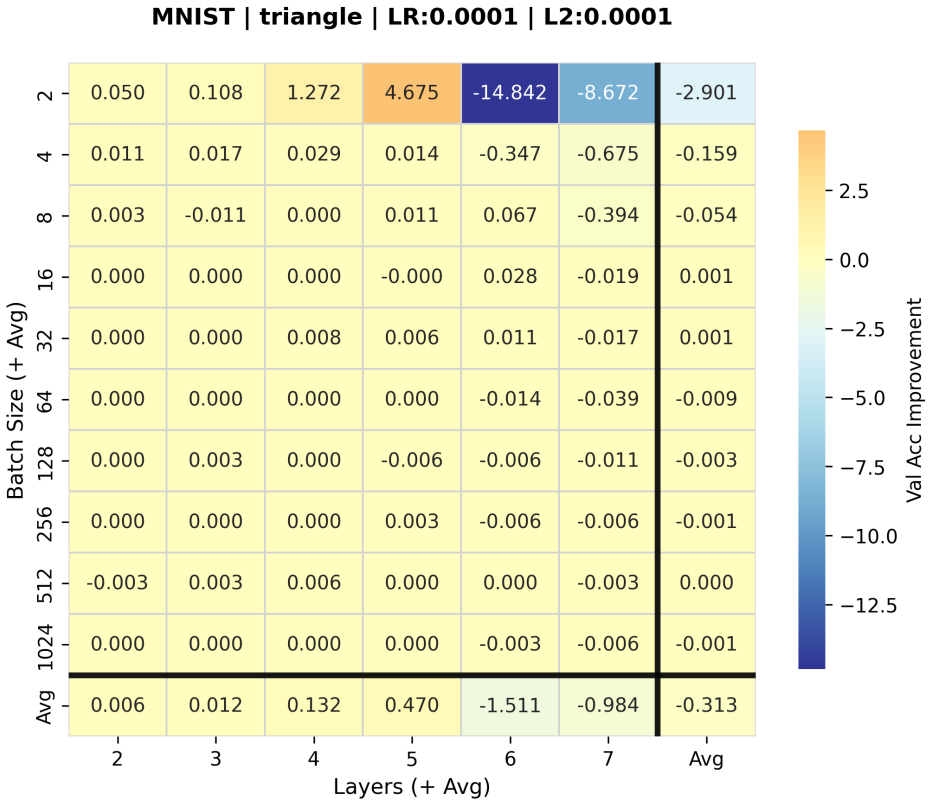
Table 2: Top 20 Configurations Sorted by Mean Improvement

Batch	Layers	LR	L2 Decay	Architecture	Dataset	Accuracy gain mean	Std
2	6	0.0001	0.0	triangle	mnist	-14.842	5.447
2	6	0.005	0.0	triangle	mnist	-15.097	3.496
2	6	0.005	0.0001	triangle	mnist	-15.097	3.496
2	7	0.005	0.0001	persisting	fashion mnist	-17.067	2.531
2	6	0.005	0.0	persisting	mnist	-18.192	12.528
2	6	0.005	0.0001	persisting	mnist	-18.192	12.528
4	7	0.005	0.0	triangle	mnist	-18.717	32.498
4	7	0.005	0.0001	triangle	mnist	-18.717	32.498
2	7	0.005	0.0	persisting	mnist	-19.397	4.109
2	7	0.005	0.0001	persisting	mnist	-19.397	4.109
2	4	0.005	0.0001	persisting	fashion mnist	-19.714	8.89
2	5	0.005	0.0001	persisting	fashion mnist	-21.136	10.327
2	5	0.005	0.0	persisting	mnist	-24.8	15.762
2	5	0.005	0.0001	persisting	mnist	-24.8	15.762
2	6	0.005	0.0001	persisting	fashion mnist	-25.3	6.317

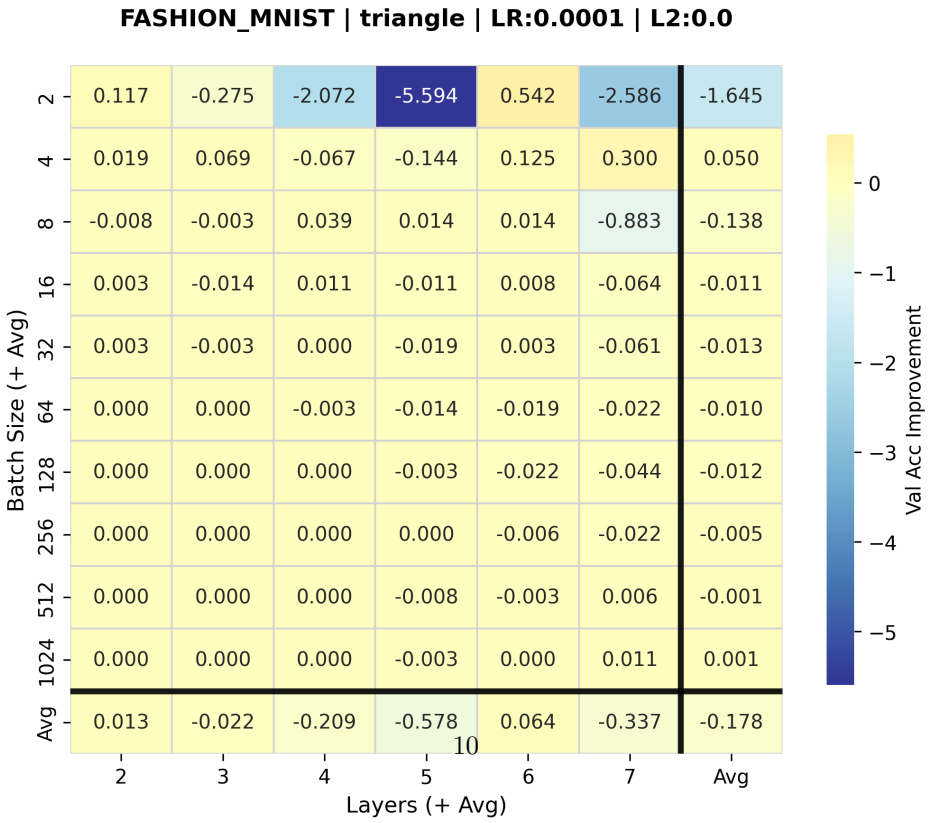
Table 3: Worst 20 Configurations Sorted by Mean Improvement

3.3 Comprehensive results heatmaps

Figure 4a, 4b and 5 show heatmaps of accuracy improvement for CIFAR-10, MNIST and Fashion MNIST respectively. The heatmaps show the best hyperparameters, averaged over the seeds. All the results are in the Appendix. For the performance gain, MNIST and Fashion MNIST require batch size equal to 2 and number of layers 5 and 6 respectively while for CIFAR-10 batch size equal to 32 yields the highest gain.



(a) Heatmap for MNIST



(b) Heatmap for Fashion-MNIST

Figure 4: Best-hyperparameter accuracy gain heatmaps for MNIST (top) and Fashion-MNIST (bottom)

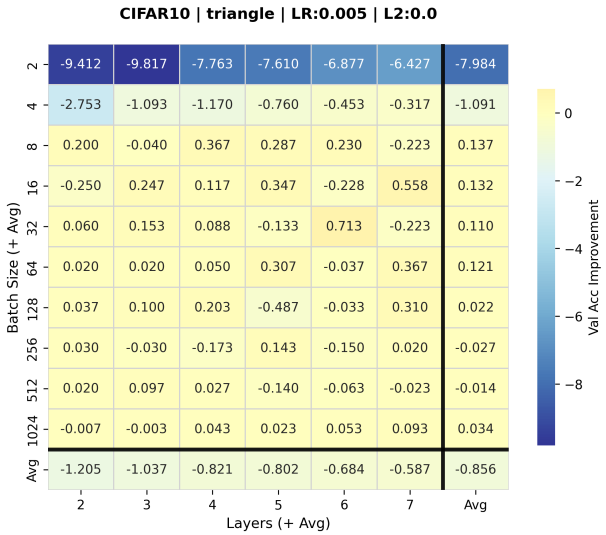


Figure 5: Best-hyperparameter accuracy gain heatmap for Cifar10

3.4 Training dynamics

Finally, the accuracy gain is robust for the mentioned specific scenarios throughout the training process. The online update method also decreases the loss during the training. Figure 6 shows the training logs for both train and validation subsets over epochs; different line styles correspond to different seeds. Both loss decrease and metric gain are without large fluctuations during the training for the new algorithm.

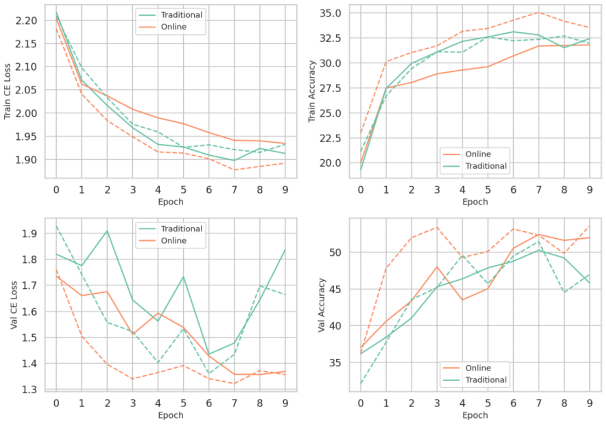


Figure 6: Training logs for MNIST, triangle architecture, LR 0.001, L2 0.0001

4 Discussion

4.1 Key Findings

Updating weights during backpropagation changes the accuracy metric in 55% of experiments by less than 0.1%, improves in 10% and decreases in 35%. Dataset complexity affects the specific combinations of hyperparameters giving deviation from the traditional approach. Even the proof-of-concept (PoC) algorithm gave 4% accuracy gain for MNIST and 0.7% for CIFAR-10.

4.2 Methods mechanism explanation

The performance gain cannot be explained in the same way as the original Nesterov Accelerated Gradient method. NAG is the optimizer and performs gradient descent steps - the optimizer is not connected with the chain rule and the dLoss forwarding to the previous layers.

Weights update during backpropagation involves gradient evaluation at different points of the weight space. This mixing of network states is caused by multiplying the dLoss by the new updated weights matrix while the dLoss was obtained during the forward pass with the old weights matrix. In classical backpropagation, all gradients are computed at a single network state at the same time — the weights remain fixed throughout the backward pass, Figure 1. In other words, all gradients are computed at one point, for the same model state.

In contrast, the online approach leads to dynamic gradient computation process where the loss gradients (dLoss) are propagated using weights that have been updated during the same backward pass, Figure 2. To define the effect of the states mixing of the network, one should analyze deeply the difference between the weight matrices and how bias propagates to early layers. Evaluating gradients not at the point where the forward pass was performed could be similar to injecting noise in gradients.

Neelakantan et al. 2015 [55] shows that adding noise into the gradients gives better convergence. Noisy gradients cause better convergence because noise helps to escape from a local minimum. Pure SGD with small batch size works better than the Adam-family methods under certain conditions [56] because of the same reason - noise helps to escape poor local minimums. In comparison with [55], the proposed algorithm has the same computational complexity as the

original backpropagation algorithm: we only rearranging operations and do not performing matrix addition. Finally, additional research is necessary to determine, how mixing the online approach influences the training process, why improves it and how to increase gains.

4.3 Implications

Regardless of shallow theoretical explanations, these observed gains show possible practical implications. The algorithm can be readily implemented in existing simple pipelines to evaluate potential performance gains. As we showed, the new method can yield up to 4% gain which is worthwhile given the same computational complexity. Speaking about the scientific implications, this work can lead to a series of following works, figuring out how the online backpropagation could enhance the training process in deep learning in a more stable way.

4.4 Limitations

The experimental scope of the current study is limited by architectural and hyperparameter diversity, dataset complexity and the proposed algorithm's parameters. Only simple multilayer perceptrons were tested with the small set of hyperparameters. The most promising extension is to perform online weight updates using momentum as in the Nesterov Accelerate Gradient method. EMA can be calculated between a weight matrix used for the forward pass and a new updated weight matrix. Another option is to use not SGD but more advanced optimizers like Adam. Also, testing the approach for convolutional networks or transformers with bigger datasets could reveal some improvements. One can explore combinations of the traditional and new approaches during the training process: apply the new method with probability or apply only for certain layers, on certain epochs, adjusting parameters of EMA depending on the depth of the layer. Hybrid and probabilistic strategies could act as regularizers, potentially yielding consistent gains.

4.5 Conclusions

To conclude, further theoretical and experimental work is needed to establish mathematical foundations and optimal conditions for weight update during backpropagation. Such research could potentially boost state-of-the-art algorithms in deep learning.

References

[1] Zhong-Qiu Zhao et al. "Object detection with deep learning: A review". In: *IEEE transactions on neural networks and learning systems* 30.11 (2019), pp. 3212–3232.

[2] Zhengxia Zou et al. "Object detection using convolutional neural networks and transformer-based models: a review". In: *Journal of Electrical Systems and Information Technology* 10.1 (2023), pp. 1–21.

[3] Zhenqiang Li et al. "A comprehensive review of computer vision techniques: datasets, methods and applications". In: *Pattern Recognition* 109 (2021), p. 107595.

[4] Shaoqing Ren et al. "Faster r-cnn: Towards real-time object detection with region proposal networks". In: *Advances in neural information processing systems*. 2015, pp. 91–99.

[5] Jonathan Long, Evan Shelhamer, and Trevor Darrell. "Fully convolutional networks for semantic segmentation". In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2015, pp. 3431–3440.

[6] Alexander Kirillov et al. "Panoptic segmentation". In: (2019), pp. 9404–9413.

[7] Lukas Ruff et al. "A unifying review of deep and shallow anomaly detection". In: *Proceedings of the IEEE* 109.5 (2021), pp. 756–795.

[8] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. "Neural machine translation by jointly learning to align and translate". In: *arXiv preprint arXiv:1409.0473* (2014).

[9] Yonghui Wu et al. "Google's neural machine translation system: Bridging the gap between human and machine translation". In: *arXiv preprint arXiv:1609.08144* (2016).

[10] Alec Radford et al. "Language models are unsupervised multitask learners". In: *OpenAI blog* 1.8 (2019), p. 9.

[11] Tom Brown et al. "Language models are few-shot learners". In: *Advances in neural information processing systems* 33 (2020), pp. 1877–1901.

[12] Jonathan Mallinson, Rico Sennrich, and Mirella Lapata. "Paraphrasing revisited with neural machine translation". In: *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 1, Long Papers*. 2017, pp. 881–893.

[13] Yogesh Balaji et al. "eDiff-I: Text-to-image diffusion models with an ensemble of expert denoisers". In: *arXiv preprint arXiv:2211.01324* (2022).

[14] Robin Rombach et al. "High-resolution image synthesis with latent diffusion models". In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2022, pp. 10684–10695.

[15] Junnan Li et al. "Blip: Bootstrapping language-image pre-training for unified vision-language understanding and generation". In: *International conference on machine learning*. PMLR. 2022, pp. 12888–12900.

[16] Jianfeng Wang et al. "Git: A generative image-to-text transformer for vision and language". In: *arXiv preprint arXiv:2205.14100*. 2022.

[17] Ian Goodfellow et al. "Generative adversarial nets". In: *Advances in neural information processing systems* 27 (2014).

[18] Antonia Creswell et al. "Generative adversarial networks: An overview". In: *IEEE Signal Processing Magazine* 35.1 (2018), pp. 53–65.

[19] Prafulla Dhariwal and Alex Nichol. "Diffusion models beat gans on image synthesis". In: *Advances in Neural Information Processing Systems* 34 (2021), pp. 8780–8794.

[20] Jonathan Ho, Ajay Jain, and Pieter Abbeel. “Denoising diffusion probabilistic models”. In: *Advances in neural information processing systems* 33 (2020), pp. 6840–6851.

[21] Jiaming Song, Chenlin Meng, and Stefano Ermon. “Denoising diffusion implicit models”. In: *arXiv preprint arXiv:2010.02502* (2021).

[22] Alexei Baevski et al. “wav2vec 2.0: A framework for self-supervised learning of speech representations”. In: *Advances in neural information processing systems* 33 (2020), pp. 12449–12460.

[23] Anmol Gulati et al. “Conformer: Convolution-augmented transformer for speech recognition”. In: *arXiv preprint arXiv:2005.08100*. 2020.

[24] Kai Shen et al. “Naturalspeech 2: Latent diffusion models are natural and zero-shot speech and singing synthesizers”. In: *arXiv preprint arXiv:2304.09116* (2023).

[25] Yuxuan Wang et al. “Tacotron: Towards end-to-end speech synthesis”. In: *arXiv preprint arXiv:1703.10135*. 2017.

[26] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. “Deep learning”. In: *nature* 521.7553 (2015), pp. 436–444.

[27] Yoshua Bengio, Aaron Courville, and Pascal Vincent. “Representation learning: A review and new perspectives”. In: *IEEE transactions on pattern analysis and machine intelligence* 35.8 (2013), pp. 1798–1828.

[28] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT press, 2016.

[29] Vinod Nair and Geoffrey E Hinton. “Rectified linear units improve restricted boltzmann machines”. In: *Proceedings of the 27th international conference on machine learning (ICML-10)*. 2010, pp. 807–814.

[30] Prajit Ramachandran, Barret Zoph, and Quoc V Le. “Searching for activation functions”. In: *arXiv preprint arXiv:1710.05941*. 2017.

[31] Diganta Misra. “Mish: A self regularized non-monotonic activation function”. In: *arXiv preprint arXiv:1908.08681* (2019).

[32] Yann N Dauphin et al. “Language modeling with gated convolutional networks”. In: *International conference on machine learning*. PMLR. 2017, pp. 933–941.

[33] Sergey Ioffe and Christian Szegedy. “Batch normalization: Accelerating deep network training by reducing internal covariate shift”. In: *International conference on machine learning*. PMLR. 2015, pp. 448–456.

[34] Yuxin Wu and Kaiming He. “Group normalization”. In: *Proceedings of the European conference on computer vision (ECCV)*. 2018, pp. 3–19.

[35] Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. “Instance normalization: The missing ingredient for fast stylization”. In: *arXiv preprint arXiv:1607.08022*. 2016.

[36] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. “Layer normalization”. In: *arXiv preprint arXiv:1607.06450*. 2016.

[37] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. “Learning representations by back-propagating errors”. In: *nature* 323.6088 (1986), pp. 533–536.

[38] Venkatesh Balavadhani Parthasarathy et al. “The Ultimate Guide to Fine-Tuning LLMs from Basics to Breakthroughs: An Exhaustive Review of Technologies, Research, Best Practices, Applied Research Challenges and Opportunities”. In: *arXiv preprint arXiv:2408.13296* (2024). https://arxiv.org/abs/2408.13296.

[39] Ashish Vaswani et al. “Attention is all you need”. In: *Advances in neural information processing systems* 30 (2017).

[40] Joel Lamy-Poirier. “Layered gradient accumulation and modular pipeline parallelism: fast and efficient training of large language models”. In: *arXiv preprint arXiv:2106.02679* (2021). https://arxiv.org/abs/2106.02679.

[41] Arvind Neelakantan et al. “Adding Gradient Noise Improves Learning for Very Deep Networks”. In: *International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=rkjZ 2016.

[42] Ilya Loshchilov and Frank Hutter. “Decoupled Weight Decay Regularization”. In: *arXiv preprint arXiv:1711.05101* (2019). URL: https://arxiv.org/abs/1711.05101.

[43] Diederik P Kingma and Jimmy Ba. “Adam: A method for stochastic optimization”. In: *arXiv preprint arXiv:1412.6980* (2014).

[44] Tijmen Tieleman and Geoffrey Hinton. *Lecture 6.5 – RMSProp: Divide the gradient by a running average of its recent magnitude*. Coursera: Neural Networks for Machine Learning. Lecture 6.5, slide 29. 2012. URL: https://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf.

[45] Anonymous (double-blind review). “When, Why and How Much? Adaptive Learning Rate Scheduling by Monitoring Training Dynamics”. In: *International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=1JPfHljXL4. 2024.

[46] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. “On the difficulty of training recurrent neural networks”. In: *International Conference on Machine Learning*. 2013, pp. 1310–1318.

[47] Christian Szegedy et al. “Rethinking the Inception Architecture for Computer Vision”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2016, pp. 2818–2826.

[48] Yurii Nesterov. “A method of solving a convex programming problem with convergence rate O(1/k$\hat{2}$)”. In: *Soviet Mathematics Doklady* 27.2 (1983), pp. 372–376.

[49] Boris T. Polyak. “Some methods of speeding up the convergence of iteration methods”. In: *USSR Computational Mathematics and Mathematical Physics* 4.5 (1964), pp. 1–17.

[50] Kaiming He et al. “Delving deep into rectifiers: Surpassing human-level performance on imagenet classification”. In: *Proceedings of the IEEE international conference on computer vision*. 2015, pp. 1026–1034.

[51] Yann LeCun et al. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.

[52] Han Xiao, Kashif Rasul, and Roland Vollgraf. “Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms”. In: *arXiv preprint arXiv:1708.07747* (2017).

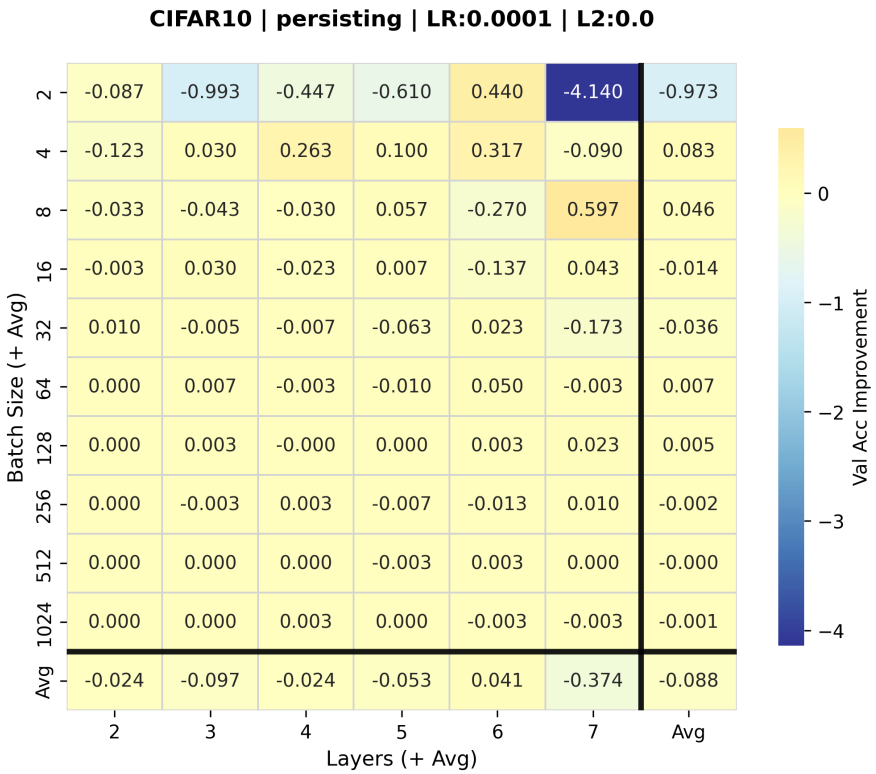
[53] Alex Krizhevsky. *Learning multiple layers of features from tiny images*. Tech. rep. University of Toronto, 2009.

[54] Adam Paszke et al. “PyTorch: An imperative style, high-performance deep learning library”. In: *Advances in neural information processing systems*. Vol. 32. 2019.

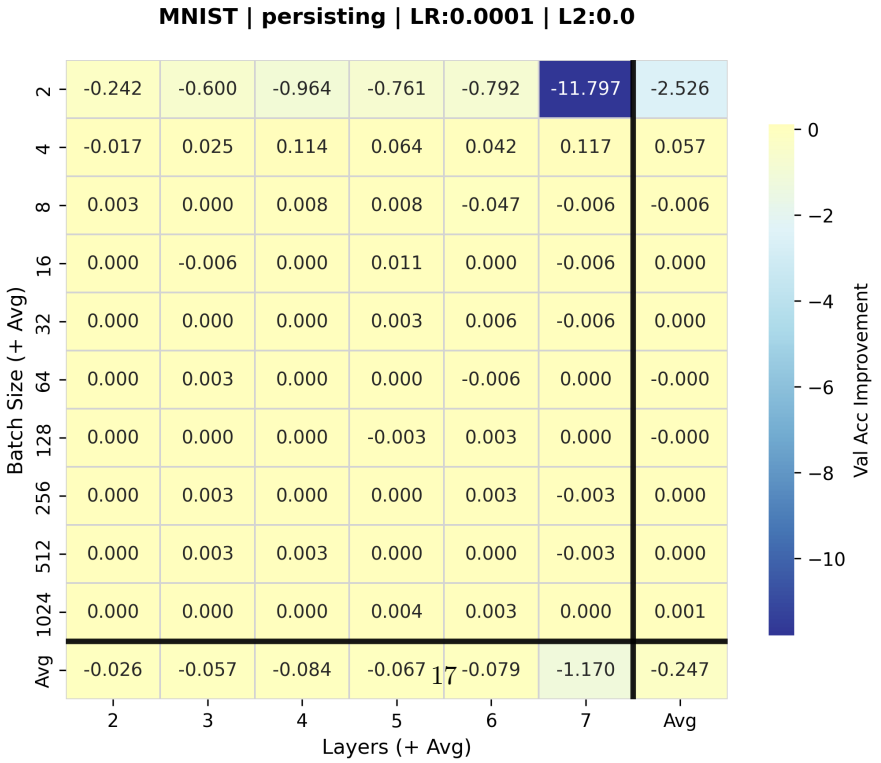
[55] Arvind Neelakantan et al. “Adding Gradient Noise Improves Learning for Very Deep Networks”. In: *arXiv preprint arXiv:1511.06807* (2015). https://arxiv.org/abs/1511.06807.

[56] Pan Zhou et al. “Towards Theoretically Understanding Why SGD Generalizes Better Than ADAM in Deep Learning”. In: *arXiv preprint arXiv:2010.05627* (2020). Submitted 12 October 2020; last revised 29 November 2021. DOI: 10.48550/arXiv.2010.05627. URL: https://arxiv.org/abs/2010.05627.

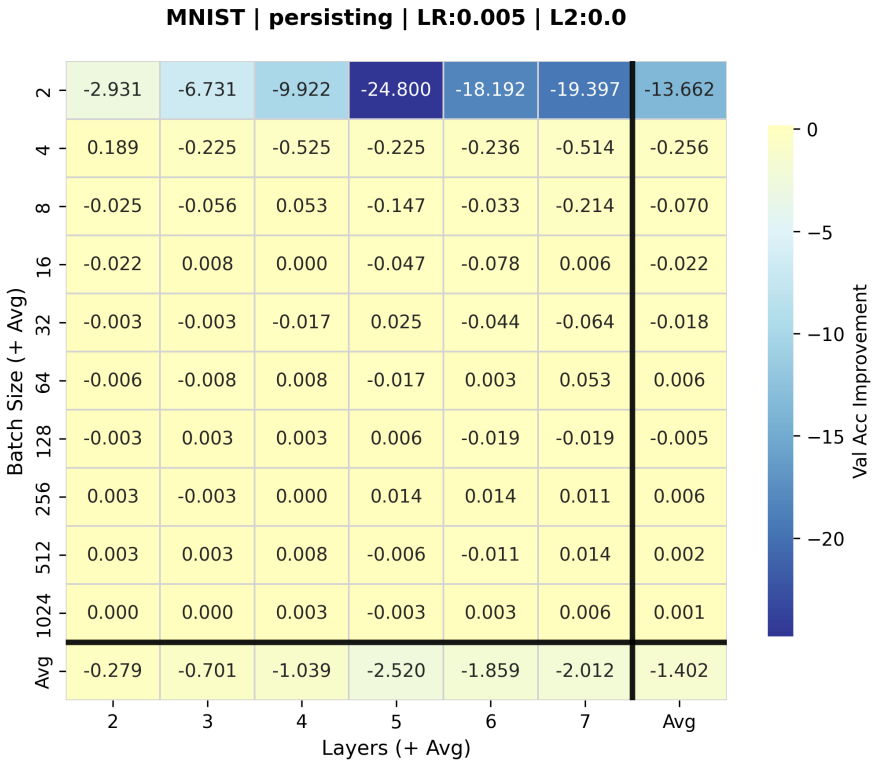
Appendix



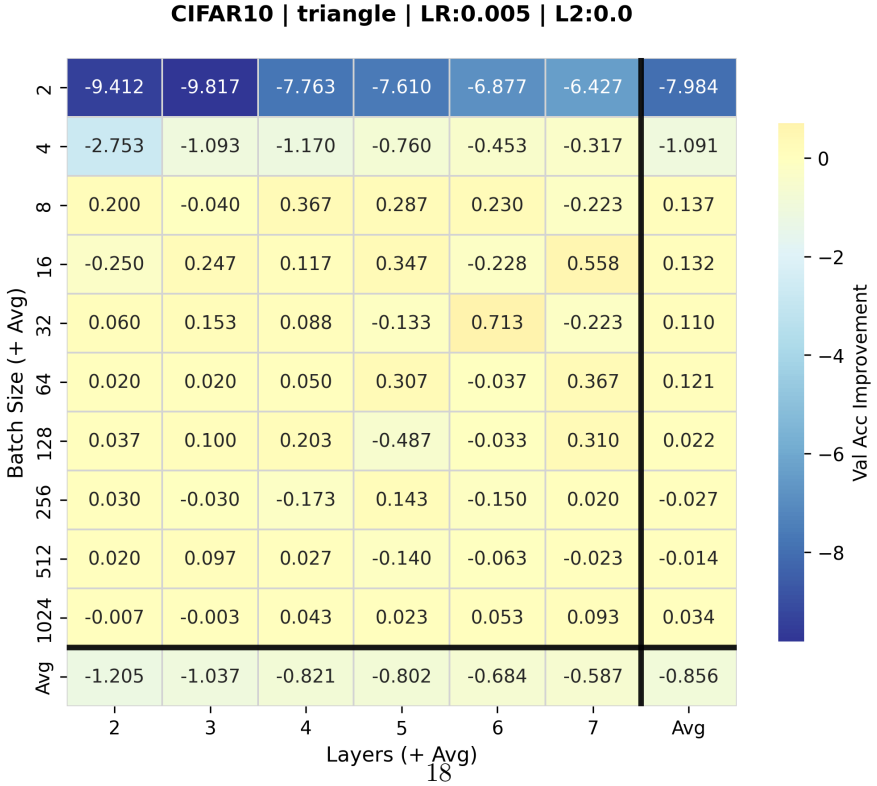
(a) CIFAR-10 | Persisting | LR: 0.0001 | L2: 0.0



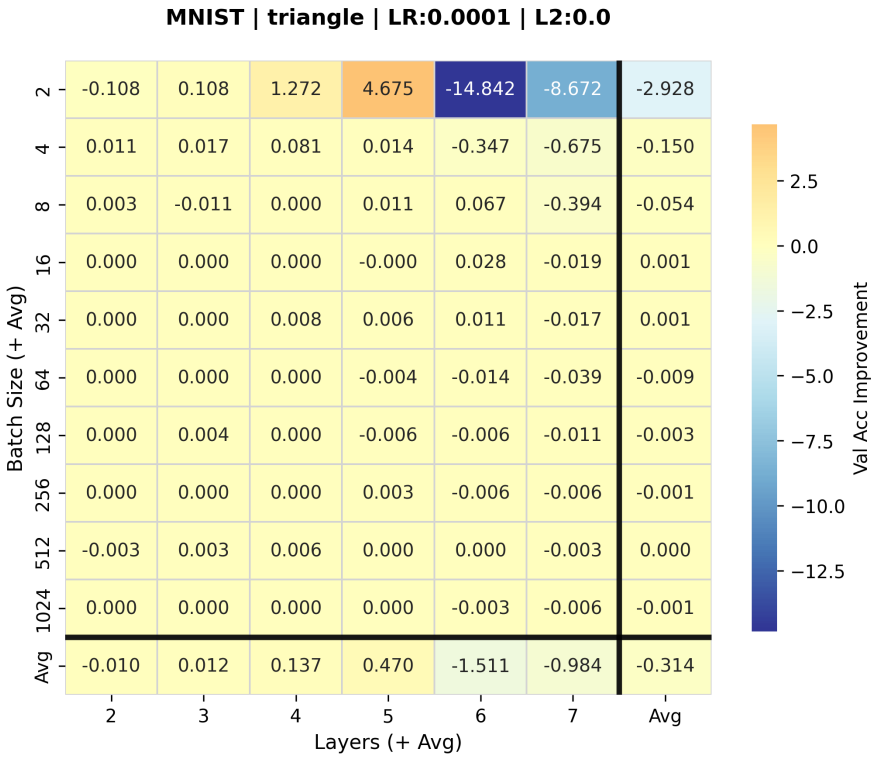
(b) MNIST | Persisting | LR: 0.0001 | L2: 0.0



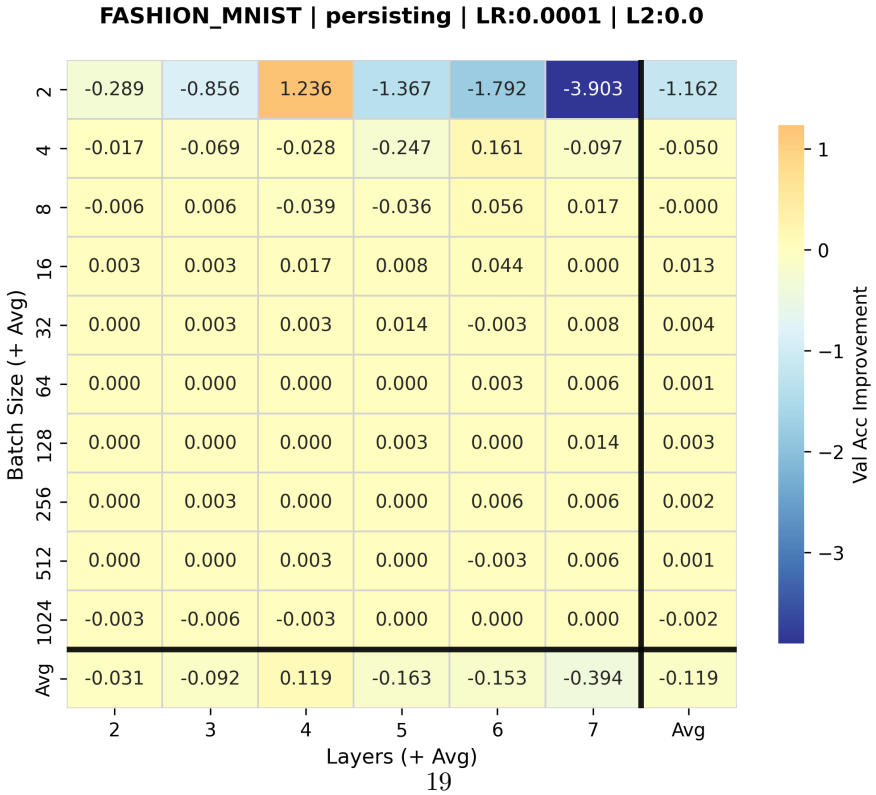
(a) MNIST | Persisting | LR: 0.005 | L2: 0.0



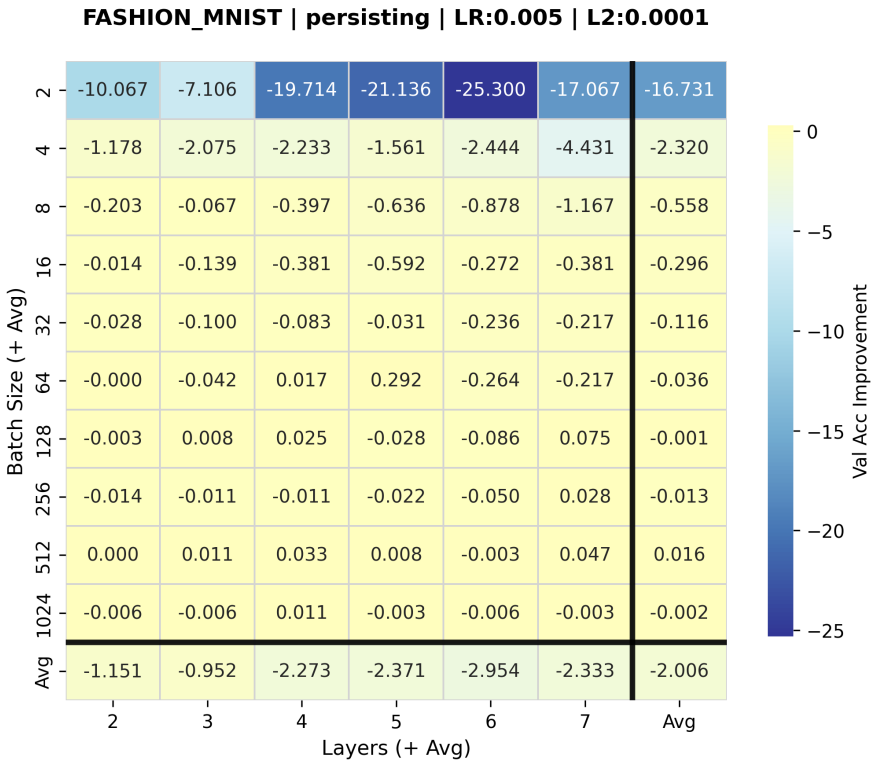
(b) CIFAR-10 | Triangle | LR: 0.005 | L2: 0.0



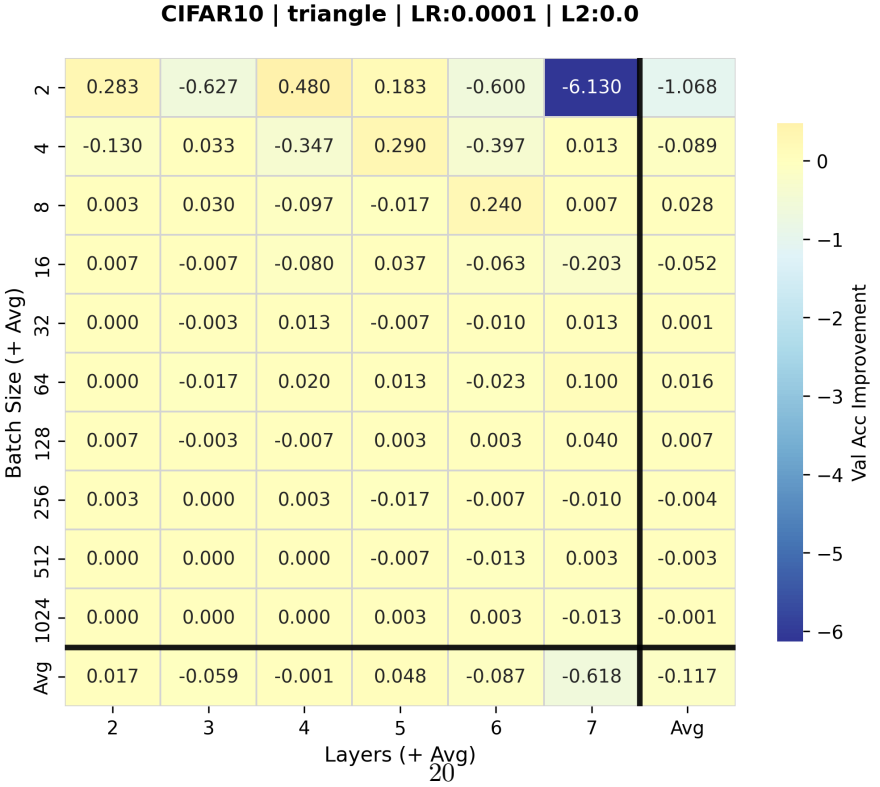
(a) MNIST | Triangle | LR: 0.0001 | L2: 0.0



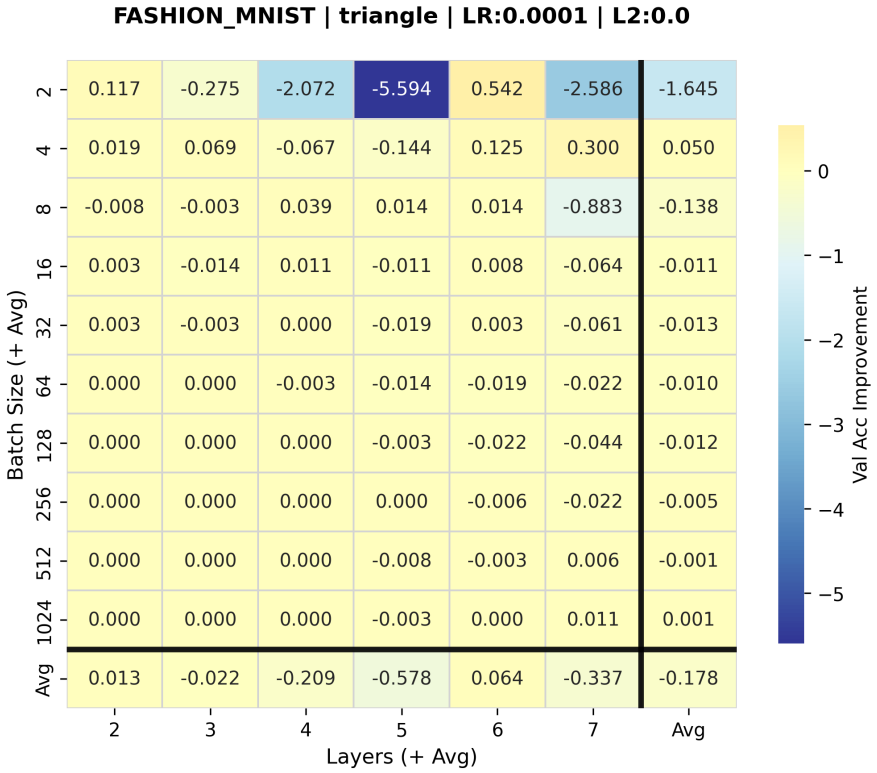
(b) Fashion-MNIST | Persisting | LR: 0.0001 | L2: 0.0



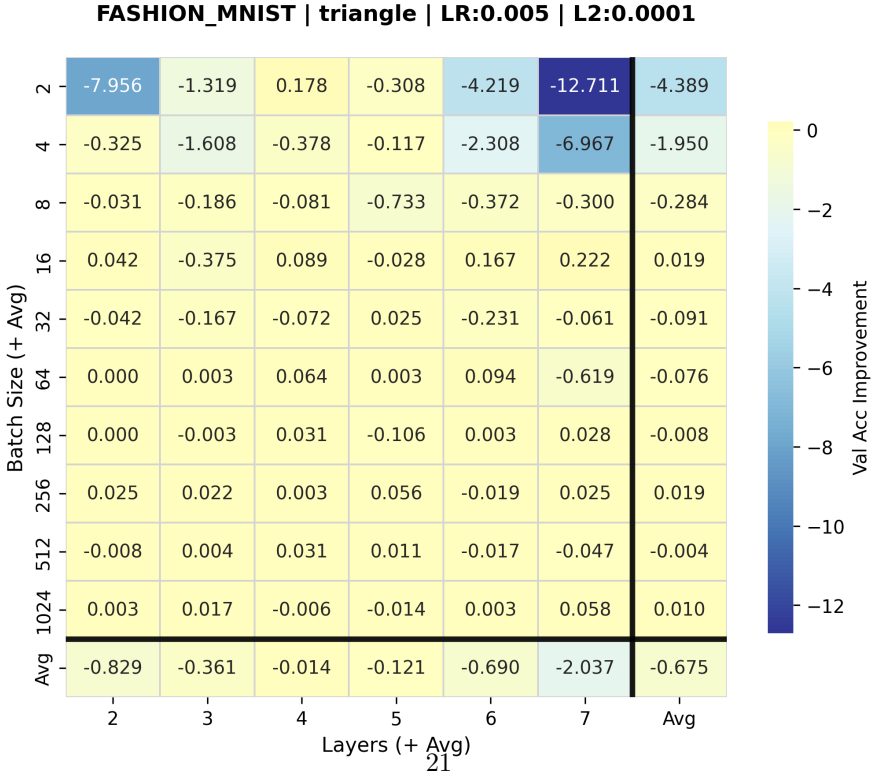
(a) Fashion-MNIST | Persisting | LR: 0.005 | L2: 0.0001



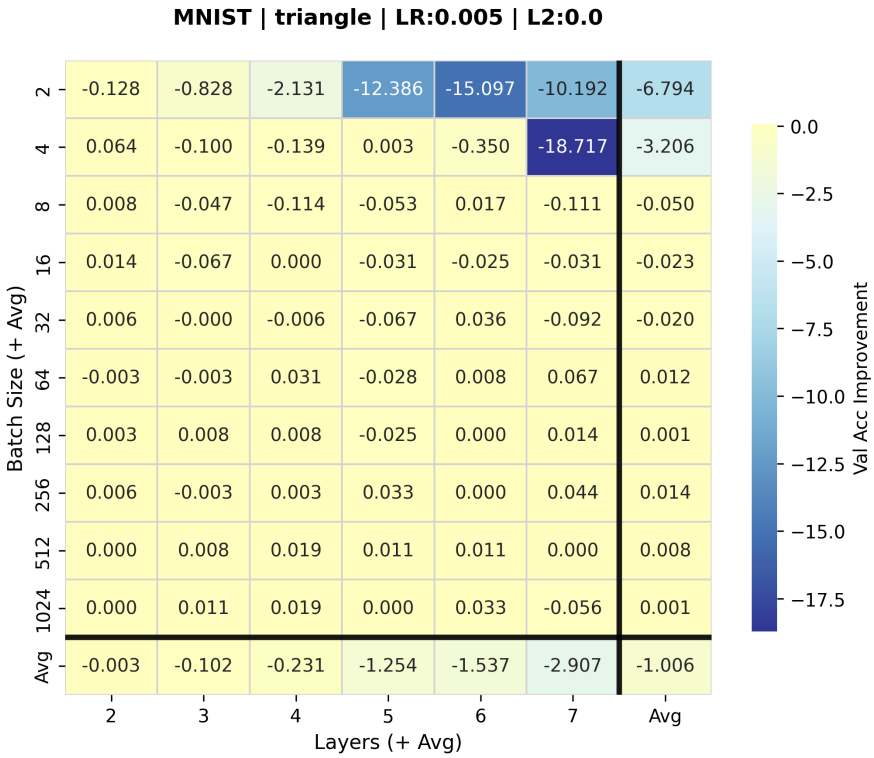
(b) CIFAR-10 | Triangle | LR: 0.0001 | L2: 0.0



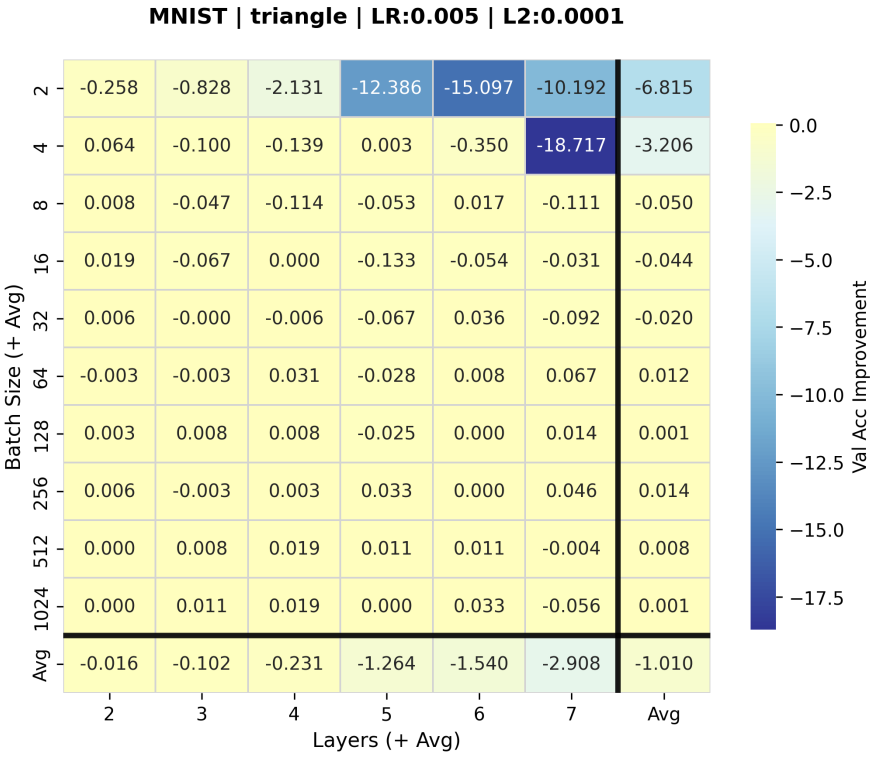
(a) Fashion-MNIST | Triangle | LR: 0.0001 | L2: 0.0



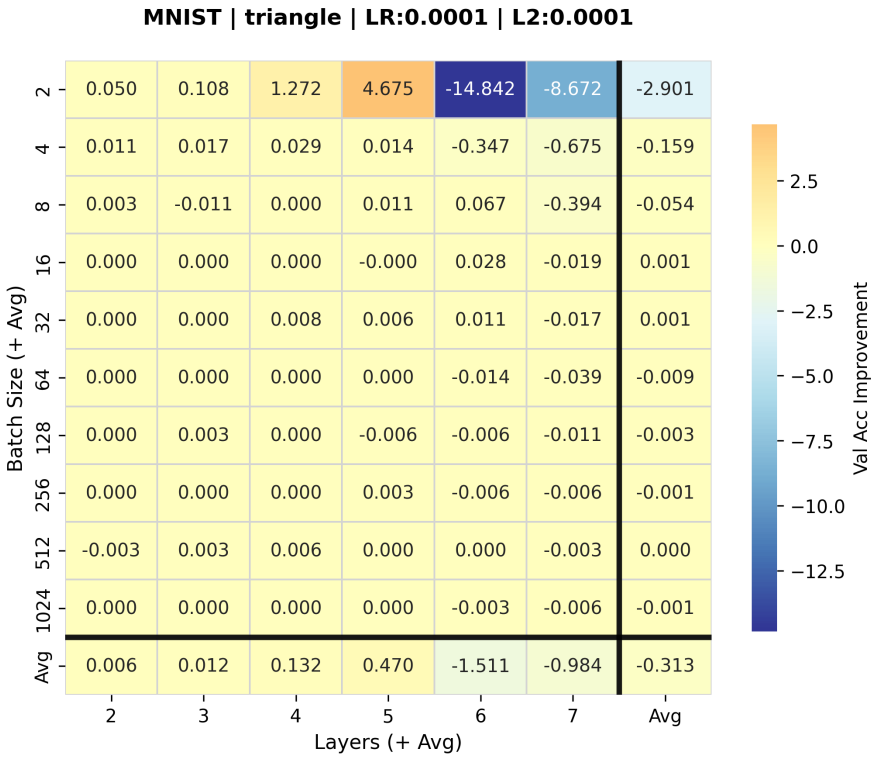
(b) Fashion-MNIST | Triangle | LR: 0.005 | L2: 0.0001



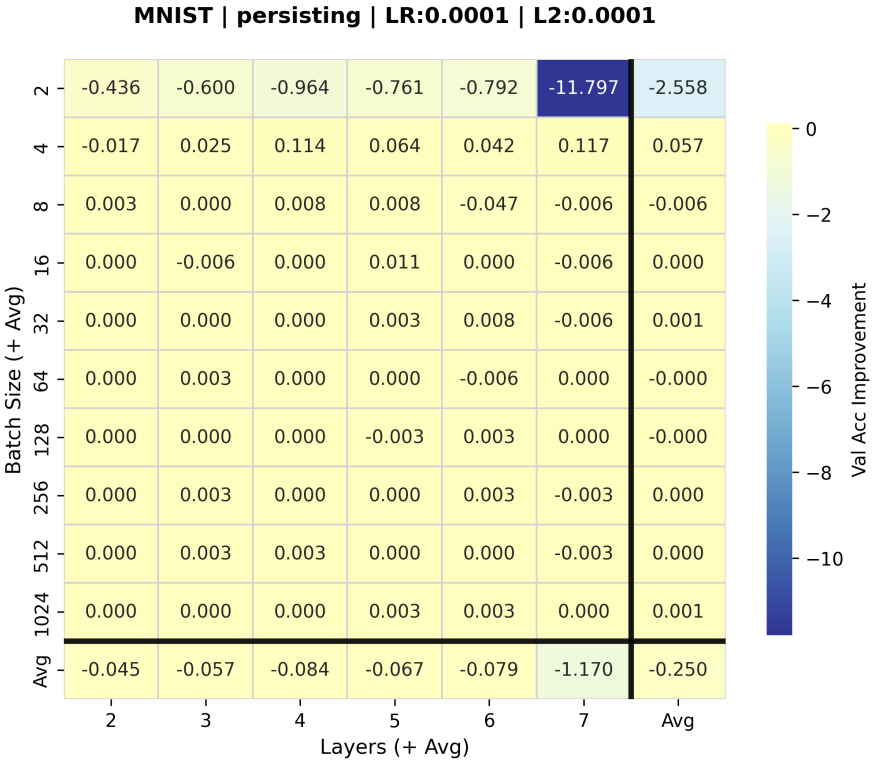
(a) MNIST | Triangle | LR: 0.005 | L2: 0.0



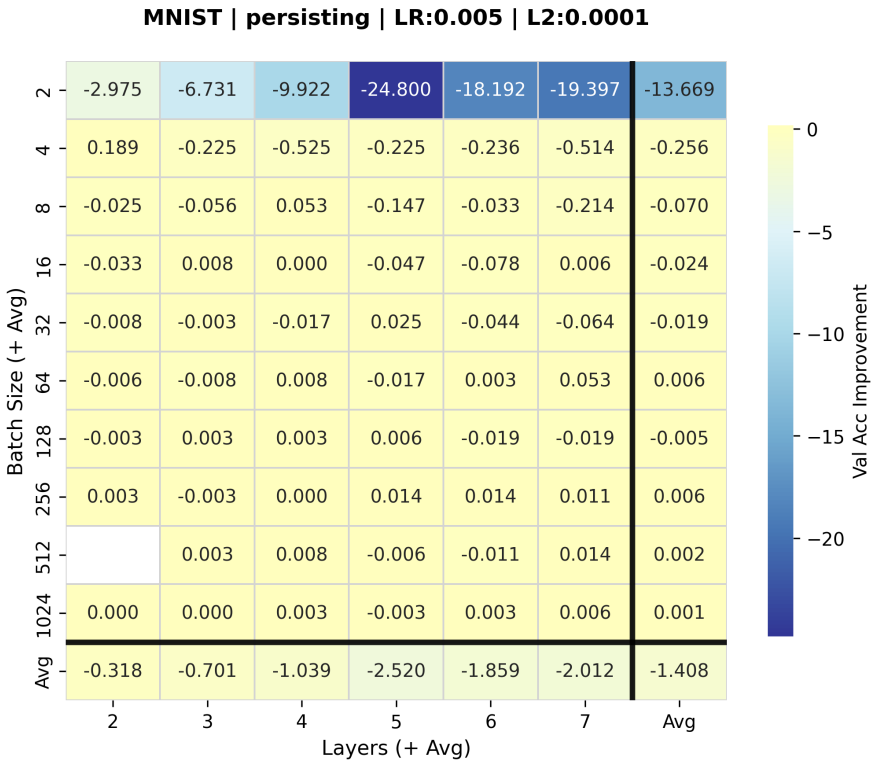
(b) MNIST | Triangle | LR: 0.005 | L2: 0.0001



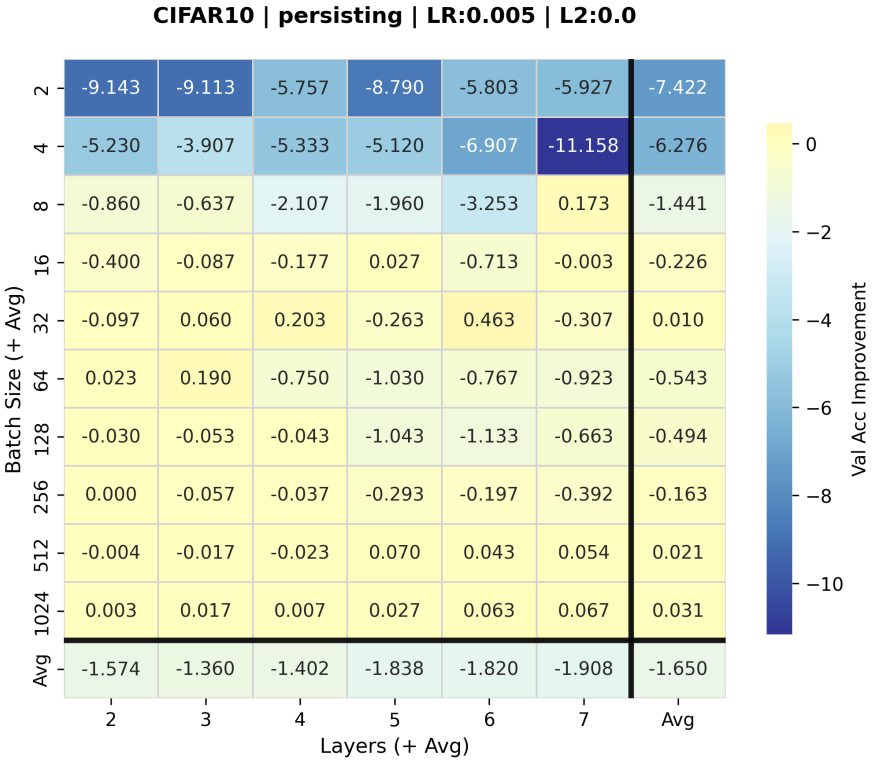
(a) MNIST | Triangle | LR: 0.0001 | L2: 0.0001



(b) MNIST | Persisting | LR: 0.0001 | L2: 0.0001



(a) MNIST | Persisting | LR: 0.005 | L2: 0.0001



(b) CIFAR-10 | Persisting | LR: 0.005 | L2: 0.0